

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Дальневосточный федеральный университет»
(ДВФУ)

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Департамент математики и компьютерного моделирования

Отчет
к лабораторной работе №4 по дисциплине «Суперкомпьютеры и параллельное программирование»

По теме: «Использование обратных асинхронных вызовов»

Вариант 20

Выполнил: студент группы Б9122-01.03.02мкт

_____ Чирич И.С.
подпись *Ф.И.О.*

Проверил: Доцент

_____ Селезнев Т.Э.
подпись *Ф.И.О*

г. Владивосток
2025

Содержание

Постановка задачи **3**

Индивидуальное задание **4**
Написание программного кода 4
Тестирование 6

Вопросы для защиты работы **8**

Выводы **9**

Постановка задачи

Цель данной лабораторной работы: изучение возможностей использования обратных асинхронных вызовов для организации ожидания завершения выполнения асинхронного метода.

Задачи лабораторной работы:

- Изучить возможности применения обратных асинхронных вызовов для параллельных потоков
- Изучить возможности асинхронного обратного вызова при использовании делегата
- Изучить возможности асинхронного обратного вызова при использовании лямбда-функции.

Индивидуальное задание

Написание программного кода

Создадим шаблон консольного проекта при помощи утилиты `dotnet`:

```
dotnet new console -n reverse_async_callback
```

В корне проекта создастся файл Project.cs, в котором и будет написан код лабораторной работы.

Создаем класс Program с методом Main:

```
class Program
{
        static void Main(string[] args)
        {

        }
}
```

Суть лабораторной работы заключается в том, чтобы реализовать логику обратного вызова после выполнения работы потока. Это осуществляется при помощи третьего параметра метода `BeginInvoke`.

В соответствии с вариантом, реализуем функцию `GetSubset`, которая по параметрам генерирует массив, затем - фильтрует его, оставляя числа, отклоняющиеся от заданного не более чем на 4:

```
static List<double> GetSubset(int size, double n)
{
        Console.WriteLine("Ассинхронные вычисления запущены.");

        const double MULTIPLIER = 1_000;
        Random random = new Random();
        List<double> list = new List<double>(size);
        for (int i = 0; i < size; i++)
        {
                list.Add(random.NextDouble() * MULTIPLIER);
        }

        List<double> newList = new List<double>();
        for (int i = 0; i < size; i++)
        {
                if (Math.Abs(list[i] - n) <= 4) newList.Add(list[i]);
        }

        Thread.Sleep(3000);
        return newList;
}
```

Для определения статуса выполнения делаем вывод в консоль сообщения о начале работы.

В методе `Main` определяем параметры для запуска `GetSubset`. Создаем делегат, и при асинхронном запуске через `BeginInvoke` передаем лямбда-функцию, в которой выводится информация о результатах вычислений:

```
static void Main(string[] args)
{
        Console.WriteLine("Главный поток начал работу. ");
        const int SIZE = 1_000_000;
        const double TARGET_VALUE = 234.5;
        Func<int, double, List<double> > dl = GetSubset;
        dl.BeginInvoke(SIZE, TARGET_VALUE, (ar) =>
        {
                if (ar == null) throw new ArgumentNullException("\'ar\' takes the value
                     null");
                Func<int, double, List<double>> getted_dl = ar.AsyncState as Func<int,
                    double, List<double>>;
                // it`s equal to: dl (4'th argument in BeginInvoke)

                List<double> result = getted_dl.EndInvoke(ar);
                Console.WriteLine("Ассинхронные вычисления выполнены.");
                Console.WriteLine("Длина сгенерированного массива: {0}", SIZE);
                Console.WriteLine("Длина полученного массива: {0}", result.Count);
        }, dl);
        Console.WriteLine("Главный поток завершил работу. Ожидание ассинхронного
            делегата...");
        Console.ReadKey(); // Waiting any key for wait the delegate
}
```

Отметим, что после завершения основного потока программа завершает выполнение, не дожидаясь выполнения асинхронного метода. Чтобы увидеть результат, добавим в конце `Console.ReadKey()`, что позволит приостановить главный поток.

Итоговый код в листинге ниже:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace reverse_async_callback
{
        internal class Program
        {
                static void Main(string[] args)
                {
                        Console.WriteLine("Главный поток начал работу. ");
                        const int SIZE = 1_000_000;
                        const double TARGET_VALUE = 234.5;
                        Func<int, double, List<double> > dl = GetSubset;
```

```
                        dl.BeginInvoke(SIZE, TARGET_VALUE, (ar) =>
                        {
                                if (ar == null) throw new ArgumentNullException("\'ar\'
                                     takes the value null");
                                Func<int, double, List<double>> getted_dl = ar.
                                    AsyncState as Func<int, double, List<double>>;
                                // it`s equal to: dl (4'th argument in BeginInvoke)

                                List<double> result = getted_dl.EndInvoke(ar);
                                Console.WriteLine("Ассинхронные вычисления выполнены.")
                                    ;
                                Console.WriteLine("Длина сгенерированного массива: {0}"
                                    , SIZE);
                                Console.WriteLine("Длина полученного массива: {0}",
                                    result.Count);
                        }, dl);
                        Console.WriteLine("Главный поток завершил работу. Ожидание
                            ассинхронного делегата...");
                        Console.ReadKey(); // Waiting any key for wait the delegate
                }

                static List<double> GetSubset(int size, double n)
                {
                        Console.WriteLine("Ассинхронные вычисления запущены.");

                        const double MULTIPLIER = 1_000;
                        Random random = new Random();
                        List<double> list = new List<double>(size);
                        for (int i = 0; i < size; i++)
                        {
                                list.Add(random.NextDouble() * MULTIPLIER);
                        }

                        List<double> newList = new List<double>();
                        for (int i = 0; i < size; i++)
                        {
                                if (Math.Abs(list[i] - n) <= 4) newList.Add(list[i]);
                        }

                        Thread.Sleep(3000);
                        return newList;
                }
        }
}
```

Тестирование

Запуск программы осуществим командой, которая скомпилирует проект и запустит его в среде выполнения:

```
dotnet build; dotnet run
```

Вывод программы:

```
Главный поток начал работу.
Главный поток завершил работу. Ожидание ассинхронного делегата...
Ассинхронные вычисления запущены.
Ассинхронные вычисления выполнены.
Длина сгенерированного массива: 1000000
Длина полученного массива: 7895
```

Программа выполняется корректно. Вывод содержит информацию о работе как основного метода, так и асинхронного метода с лямбда-функцией в качестве колбека.

Вопросы для защиты работы

1. **Поясните назначение каждого параметра метода BeginInvoke()**

   В порядке следования аргументов:

   - Аргументы делегата: то, что передается в функцию, которая будет выполняться в отдельном потоке
   - `AsyncCallback` - непосредственно колбек - функция, принимающая аргументом `IAsyncResult`.
   - `object state` - объект, который прикрепляется к колбеку. Например, если необходимо дать колбеку доступ к результатам вычислений.

2. **Почему при использовании типа делегата в качестве метода обратного вызова последний параметр метода `BeginInvoke()` можно не использовать?**

   Потому что тип `IAsyncResult` уже содержит в себе все данные о результате выполнения делегата.

3. **Опишите области использования асинхронных делегатов. В каких типах проектов .NET Framework они применимы?**

   Как пример:

   - GUI приложения, где нельзя болкировать пользовательский интерфейс (WPF / WinForms)
   - ASP.NET Web Forms
   - При создании служб Windows

Выводы

В данной лабораторной работе изучен один из способов реализации обработки асинхронных методов - колбек-функции. В частности, был продемонстрирован пример программы, в которой роль колбека выполняла лямбда-функция.